Paw Tracker

Dog Sport Tracker App

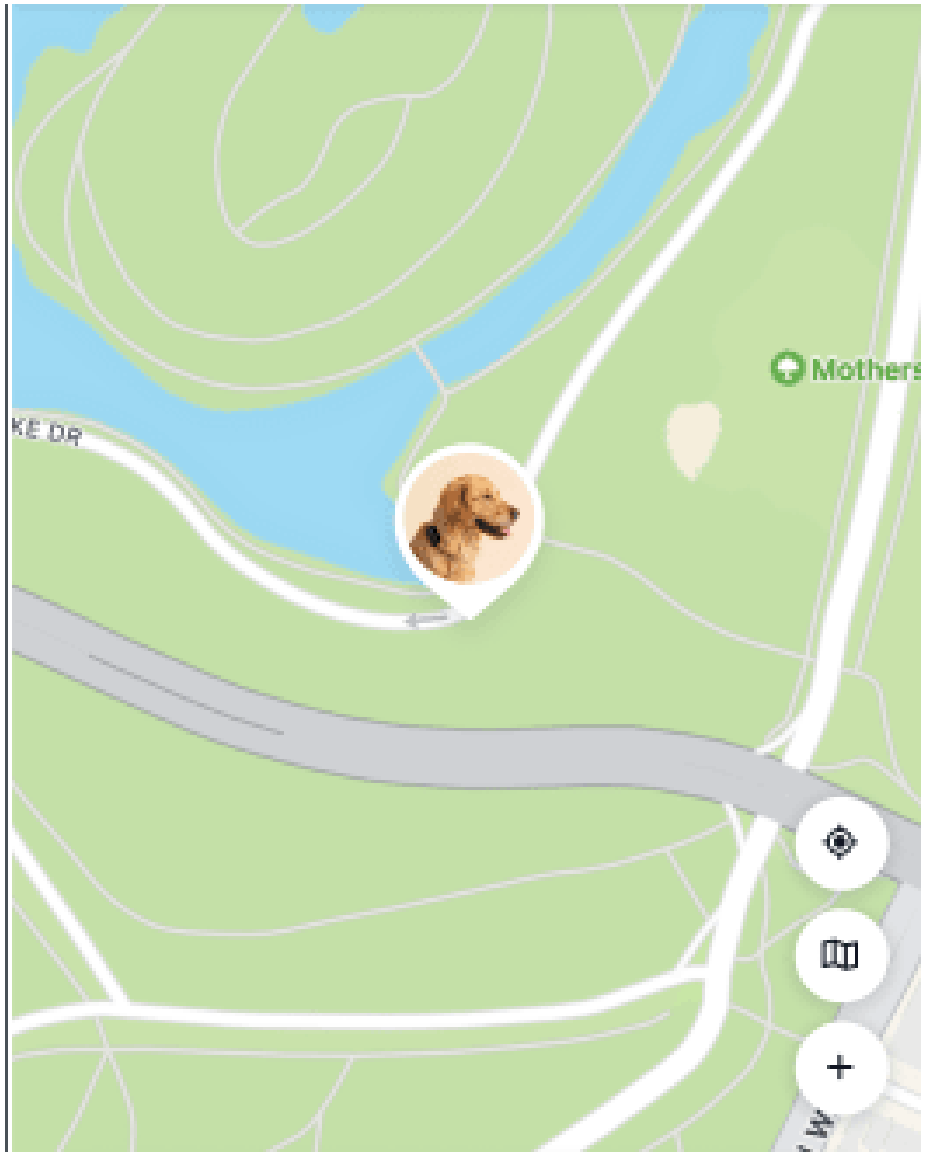
Sprint 2

Emmi Korhonen (Scrum Master)

Farnaz Jamilpanah

Sprint 2 Goals:

- Calculate distance and time
- Show recent walks
- Filter history by day or week
- Compare to recommended activity



Sprint 2 User Stories

- As a user, I want to see recent walks with distance and time so I stay motivated.
- As a user, I want to filter history by week or day.
- As a user, I want to compare runs to recommended activity, so that I know If my dog is getting enough exercise.

Jira:

Valmiit työtehtävät

Avain	Yhteenveto	Työtyyppi	Eepos	Tila
APP-2	Sprint-2-Planning	Tehtävä		VALMIS
APP-7	Sprint-2-Retrospective	Tehtävä		VALMIS
APP-56	Sprint-2-Review	Tehtävä		VALMIS
APP-23	As a user I want to see daily and weekly data of my dog's activity, so that I can review if my d...	Tehtävä		VALMIS
APP-67	As a user, I want to filter history by week or day	Tehtävä		VALMIS
APP-30	As a user I want to store my dog's previous walks, so that I can track walks on long term	Tehtävä		VALMIS
APP-65	As a user, I want to see recent walks with distance and time so I stay motivated	Tehtävä		VALMIS
APP-71	GitHub	Tehtävä		VALMIS

Sprint 2 Open Tasks

- **Goal Reached Implementation**
  - Iniitally planned for Sprint 2
  - During the sprint, it was decided to move this task to later sprint focused on statistics functionality **(Sprint 3)**

- **History Screen – Google Maps polyline**
  - Originally designed in Figma to display a small map with a polyline on the history screen
  - As this is not a high-priority feature, it has been moved to **Sprint 4 (Material Design & UI improvements**)

- **To Fix: Room database migration**

What I (Emmi) Worked On:

- WalkEntity + WalkDao + WalkRepository + AppDatabase
- TrackingViewModel stop button – save walk
- Filter logic

Room Database Configuration

```
implementation("com.google.code.gson:gson:2.10.1")
implementation(libs.room.runtime)
implementation(libs.room.ktx)
ksp(libs.room.compiler)
testImplementation(libs.junit)
androidTestImplementation(libs.androidx.junit)
```

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
}
```

WalkEntity:

```
15 Usages
@Entity(tableName = "walks", indices = [Index( ...value = "startTime")])
data class WalkEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val startTime: Long,
    val endTime: Long,
    val distance: Float,
    val duration: Long,
    val pointCount: Int,
    val previewPolyline: String? = null // Encoded polyline for Google Maps format
)
```

WalkWithPoints:

```
data class WalkWithPoints(
    @Embedded val walk: WalkEntity,

    @Relation(
        parentColumn = "id",
        entityColumn = "walkId"
    )
    val points: List<GpsPointEntity>
)
```

GPSEntity:

```
@Entity(
    tableName = "gps_points",
    foreignKeys = [
        ForeignKey(
            entity = WalkEntity::class,
            parentColumns = ["id"],
            childColumns = ["walkId"],
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [
        Index( ...value = "walkId"),
        Index(value = ["walkId", "sequence"])
    ]
)


data class GpsPointEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val walkId: Long,
    val sequence: Int,
    val timestamp: Long,
    val latitude: Double,
    val longitude: Double
)
```

AppDatabase:

```
2 Usages
@Database(entities = [WalkEntity::class, GpsPointEntity::class], version = 2)
abstract class AppDatabase : RoomDatabase() {
    1 Usage
    abstract fun walkDao(): WalkDao
}
```

WalkDao:

```
@Dao
interface WalkDao {
    1 Usage
    @Insert
    suspend fun insertWalk(walk: WalkEntity): Long

    1 Usage
    @Insert
    suspend fun insertPoints(points: List<GpsPointEntity>)

    1 Usage
    @Delete
    suspend fun deleteWalk(walk: WalkEntity)

    1 Usage
    @Query( value = "SELECT * FROM walks ORDER BY startTime DESC")
    fun getAllWalks(): Flow<List<WalkEntity>>

    1 Usage
    @Query( value = "SELECT * FROM walks WHERE startTime >= :startOfDay")
    fun getWalksFromDay(startOfDay: Long): Flow<List<WalkEntity>>

    1 Usage
    @Query( value = "SELECT * FROM walks WHERE startTime >= :startOfWeek")
    fun getWalksFromWeek(startOfWeek: Long): Flow<List<WalkEntity>>

    1 Usage
    @Transaction
    @Query( value = "SELECT * FROM walks WHERE id = :walkId")
    suspend fun getWalkWithPoints(walkId: Long): WalkWithPoints
}
```

WalkRepository:

```
class WalkRepository(private val dao: WalkDao) {

    1 Usage
    fun getAllWalks() = dao.getAllWalks()

    fun getTodayWalks() =
        dao.getWalksFromDay( startOfDay = TimeUtils.getStartOfDay())

    fun getWalksByWeek() =
        dao.getWalksFromWeek( startOfWeek = TimeUtils.getStartOfWeek())

    suspend fun getWalkDetails(walkId: Long): WalkWithPoints {
        return dao.getWalkWithPoints(walkId)
    }

    1 Usage
    suspend fun insertWalkWithPoints(
        walk: WalkEntity,
        points: List<LocationPoint>
    ) {
        // 1. Insert parent row
        val walkId = dao.insertWalk(walk)

        // 2. Convert to DB entities
        val pointEntities = points.mapIndexed { index, point ->
            GpsPointEntity(
                walkId = walkId,
                sequence = index,
                timestamp = point.time,
                latitude = point.latitude,
                longitude = point.longitude
            )
        }

        // 3. Insert all points
        dao.insertPoints(pointEntities)
    }


    suspend fun deleteWalk(walk: WalkEntity) {
```

TimeUtils:

```
3 Usages
object TimeUtils {

    1 Usage
    fun getStartOfDay(): Long {
        val calendar = Calendar.getInstance()
        calendar.set(Calendar.HOUR_OF_DAY, 0)
        calendar.set(Calendar.MINUTE, 0)
        calendar.set(Calendar.SECOND, 0)
        calendar.set(Calendar.MILLISECOND, 0)
        return calendar.timeInMillis
    }

    1 Usage
    fun getStartOfWeek(): Long {
        val calendar = Calendar.getInstance()
        calendar.set(Calendar.DAY_OF_WEEK, calendar.firstDayOfWeek)
        calendar.set(Calendar.HOUR_OF_DAY, 0)
        calendar.set(Calendar.MINUTE, 0)
        calendar.set(Calendar.SECOND, 0)
        calendar.set(Calendar.MILLISECOND, 0)
        return calendar.timeInMillis
    }
}
```

TrackingViewModel

stopTracking()

```
val points = _uiState.value.points

val walk = WalkEntity(
    startTime = startTime,
    endTime = endTime,
    distance = (_uiState.value.distance * 1000).toFloat(),
    duration = endTime - startTime,
    pointCount = points.size,
    previewPolyline = null
)

viewModelScope.launch {
    walkRepository.insertWalkWithPoints(
        walk = walk,
        points = points
    )
}
```

Filter Logic

WalkDao:

```
1 Usage
@Query( value = "SELECT * FROM walks WHERE startTime >= :startOfDay")
fun getWalksFromDay(startOfDay: Long): Flow<List<WalkEntity>>

1 Usage
@Query( value = "SELECT * FROM walks WHERE startTime >= :startOfWeek")
fun getWalksFromWeek(startOfWeek: Long): Flow<List<WalkEntity>>
```

WalkRepository:

```
fun getTodayWalks() =
    dao.getWalksFromDay( startOfDay = TimeUtils.getStartOfDay())

fun getWalksByWeek() =
    dao.getWalksFromWeek( startOfWeek = TimeUtils.getStartOfWeek())
```

TimeUtils generates time range (start of day/week) in milliseconds which are used in Room queries to filter walks by date or week

Farnaz tasks:

- Distance and Time
- HistoryScreen + HistoryUiState + HistoryViewModel
- Filter UI

```
private fun calculateDistance()
```

```
    1 Usage
    private fun calculateDistance(a: LocationPoint, b: LocationPoint): Double {
        val result = FloatArray( size = 1)
        Location.distanceBetween( startLatitude = a.latitude,  startLongitude = a.longitude,  endLatitude = b.latitude,  endLongitude = b.longitude,  results = result)
        return result[0] / 1000.0 // meters → km
    }
}
```

HistoryScreen:

```
1 Usage
@Composable
fun HistoryScreen(viewModel: HistoryViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsState()

    Column(modifier = Modifier.fillMaxSize()) {

        HistoryFilterTabs(
            selected = uiState.filter,
            onSelect = { viewModel.setFilter(it) }
        )

        HistoryList(walks = uiState.walks)
    }
}
```

```
fun HistoryFilterTabs()
```

```
fun FileterTab()
```

```
fun HistoryList()
```

```
fun WalkHistoryItem()
```

HistoryViewModel:

```
class HistoryViewModel(
    private val repository: WalkRepository
) : ViewModel() {


    5 Usages
    private val _uiState = MutableStateFlow( value = HistoryUiState())
    1 Usage
    val uiState: StateFlow<HistoryUiState> = _uiState
    init {
        loadHistory( filter = WalkFilter.Daily)
    }

    2 Usages
    private fun loadHistory(filter: WalkFilter) {
        viewModelScope.launch {
            val flow = when (filter) {
                WalkFilter.Daily -> repository.getTodayWalks()
                WalkFilter.Weekly -> repository.getWalksByWeek()
                WalkFilter.ALL -> repository.getAllWalks()

            }

            flow
                .map { list -> list.map { it.toUiModel() } }
                .collect { uiList ->
                    _uiState.value = _uiState.value.copy(walks = uiList)
                }
        }
    }


    1 Usage
    fun setFilter(filter: WalkFilter) {
        _uiState.value = _uiState.value.copy(filter = filter)
        loadHistory(filter)
```

WalkFilter

```
15 Usages
enum class WalkFilter {
    5 Usages
    Daily,
    3 Usages
    Weekly,
    1 Usage
    ALL
}
```

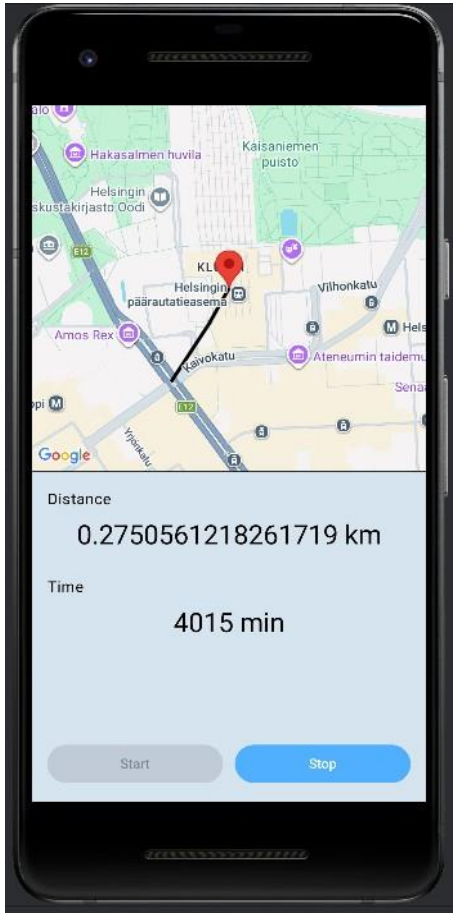
WalkUiModel:

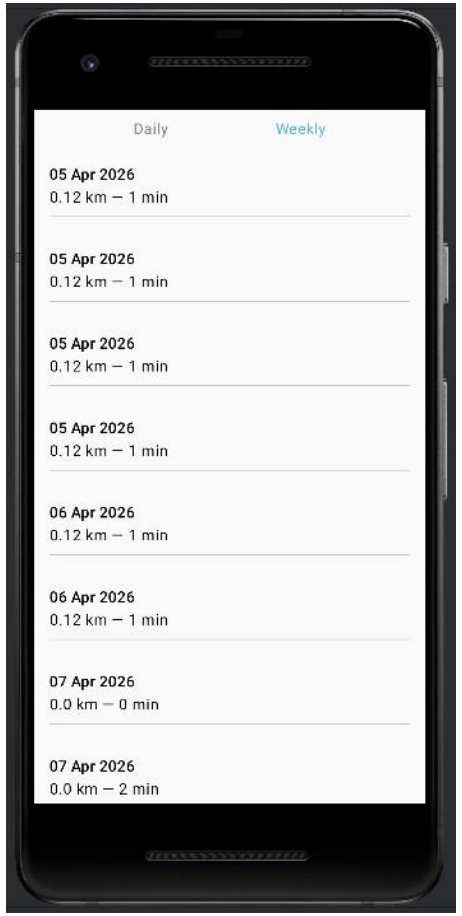
```
data class WalkUiModel(
    val id: Long,
    val date: String,
    val distanceKm: Double,
    val timeMinutes: Long

)
```

HistoryUiState:

```
data class HistoryUiState(
    val walks: List<WalkUiModel> = emptyList(),
    val filter: WalkFilter = WalkFilter.Daily
)
```





Application Demo

Retrospective

Challenges:

- **Time management:**
  - There was limited time at the end of the sprint to focus on fixing smaller details and polishing UI and the implementations.

- **Database model and overall architecture:**
  - As storing the gps points different approaches were considered. In the first implementation, the gps points were stored to a list, which was then converted into a string using type converter to fit Room database. This approach was later replaced by defining separate entity for gps points, resulting more scalable solution.

What went well:

- **Team work**
- **We successfully implemented core features for the sprint**

Sprint 2 backlog:

As a user, I want to compare runs to recommended activity, so that I know If my dog is getting enough exercise.

As a user, I want to set daily walk goals so I can stay consistent.

- **Statistics Screen**
- **Testing**
- **Fix: Clean implementation of Room database migration**

Thank you and questions